Class and sequence diagram

Sprint 1: User Authentication & Basic Website Setup

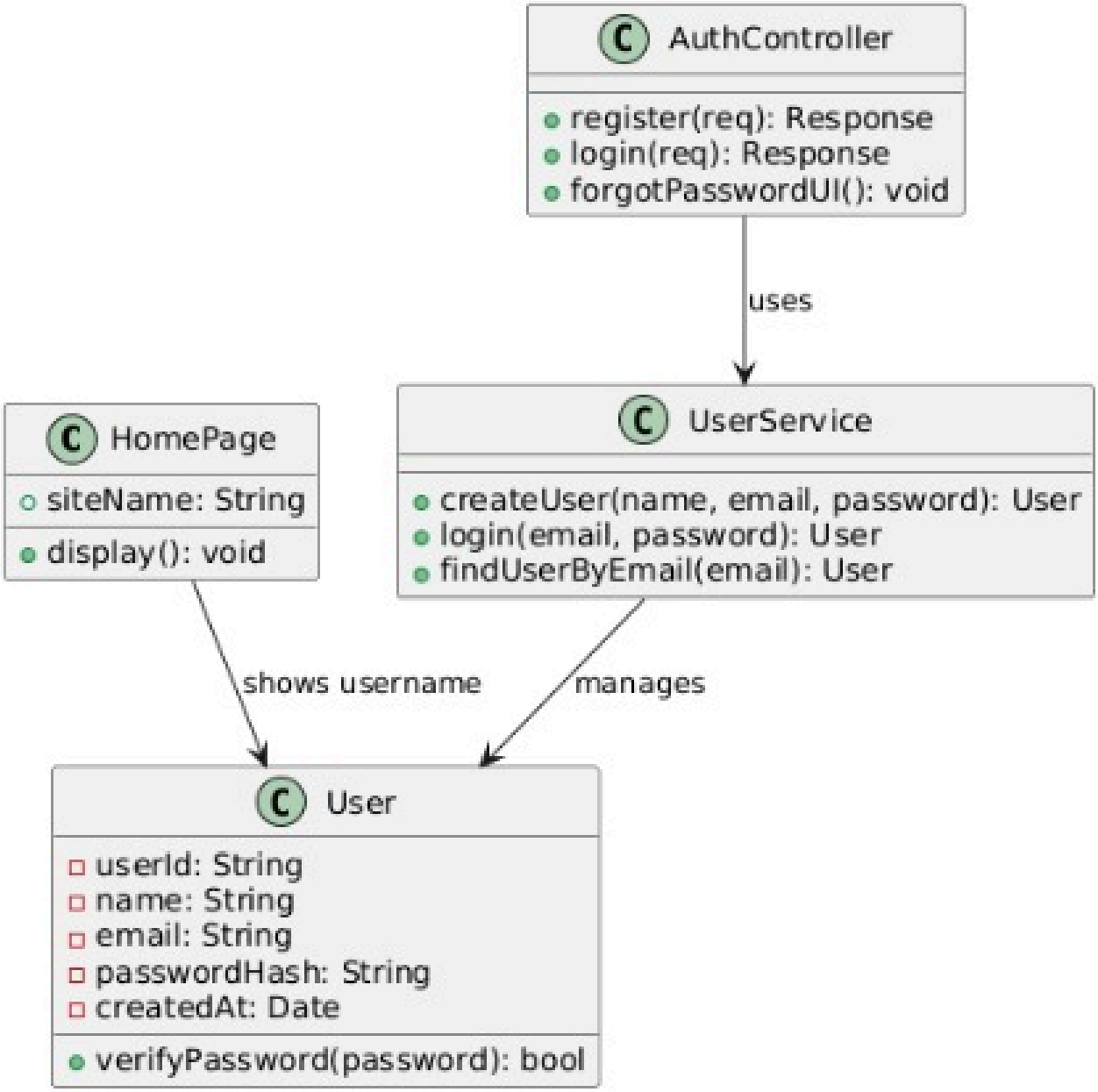
Features Implemented

1. **User Signup**
2. **User Login**
3. **Forgot Password**
4. **Basic website layout (home page, navigation)**

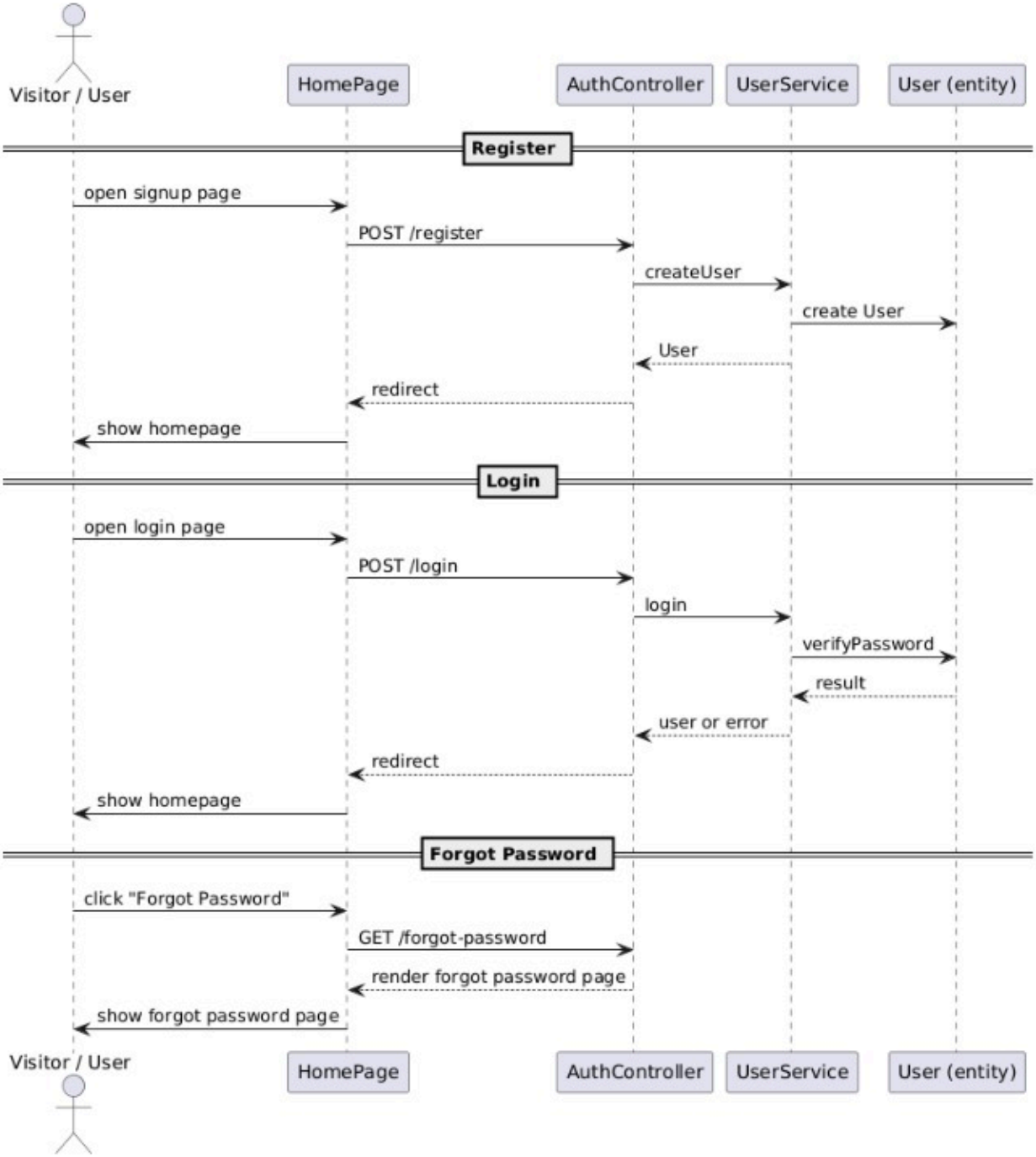
Sprint Goal

Establish the foundation of the platform with user authentication and basic upfront UI.

Class:


AuthController
register(req): Response
login(req): Response
forgotPasswordUI(): void
uses
HomePage
siteName: String
display(): void
UserService
createUser(name, email, password): User
login(email, password): User
findUserByEmail(email): User
shows username
manages
User
userId: String
name: String
email: String
passwordHash: String
createdAt: Date
verifyPassword(password): bool

Sequence:



Sprint 2:Auction Core: Create Auction, Place Bids, Auto-Bid

Features Implemented

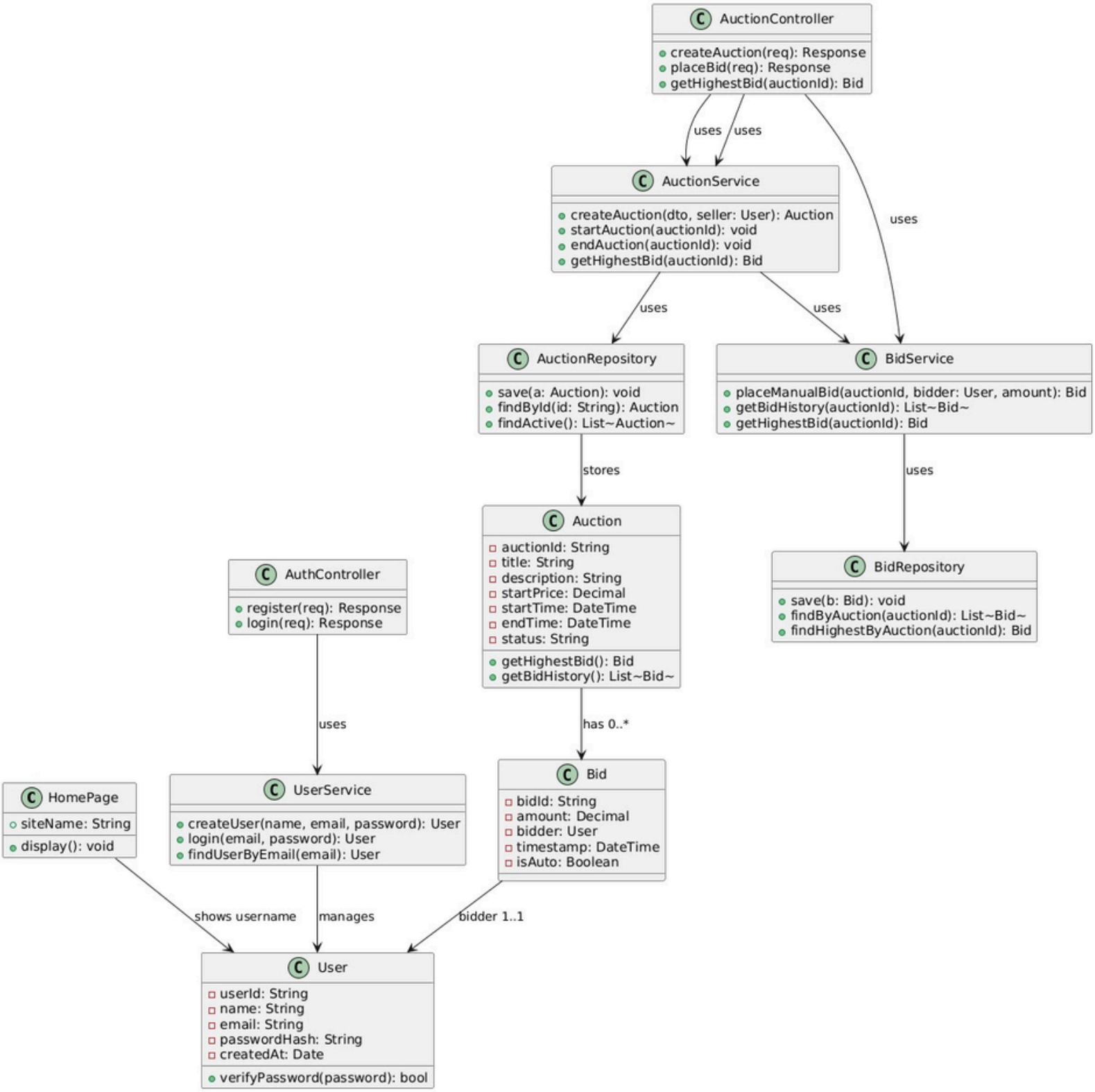
1. **Seller creates an auction**
2. **User places a bid**
3. **Auto-bid system (max bid + increment)**

4. **Basic auction detail page**

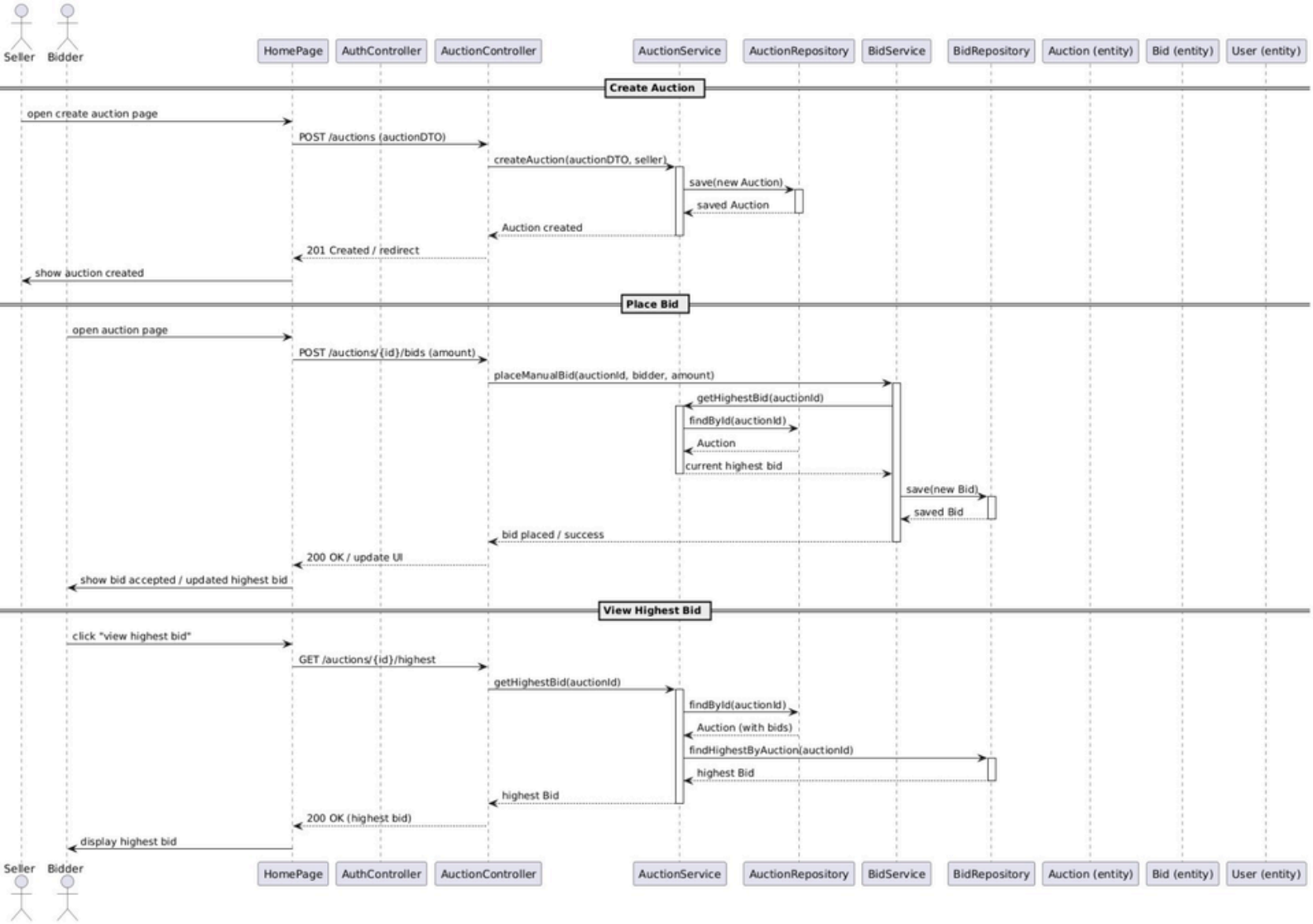
Sprint Goal

Provide complete auction creation and bidding functionality.

Class:



Sequence:



Sprint 3:Buyer Search & Auction Discovery

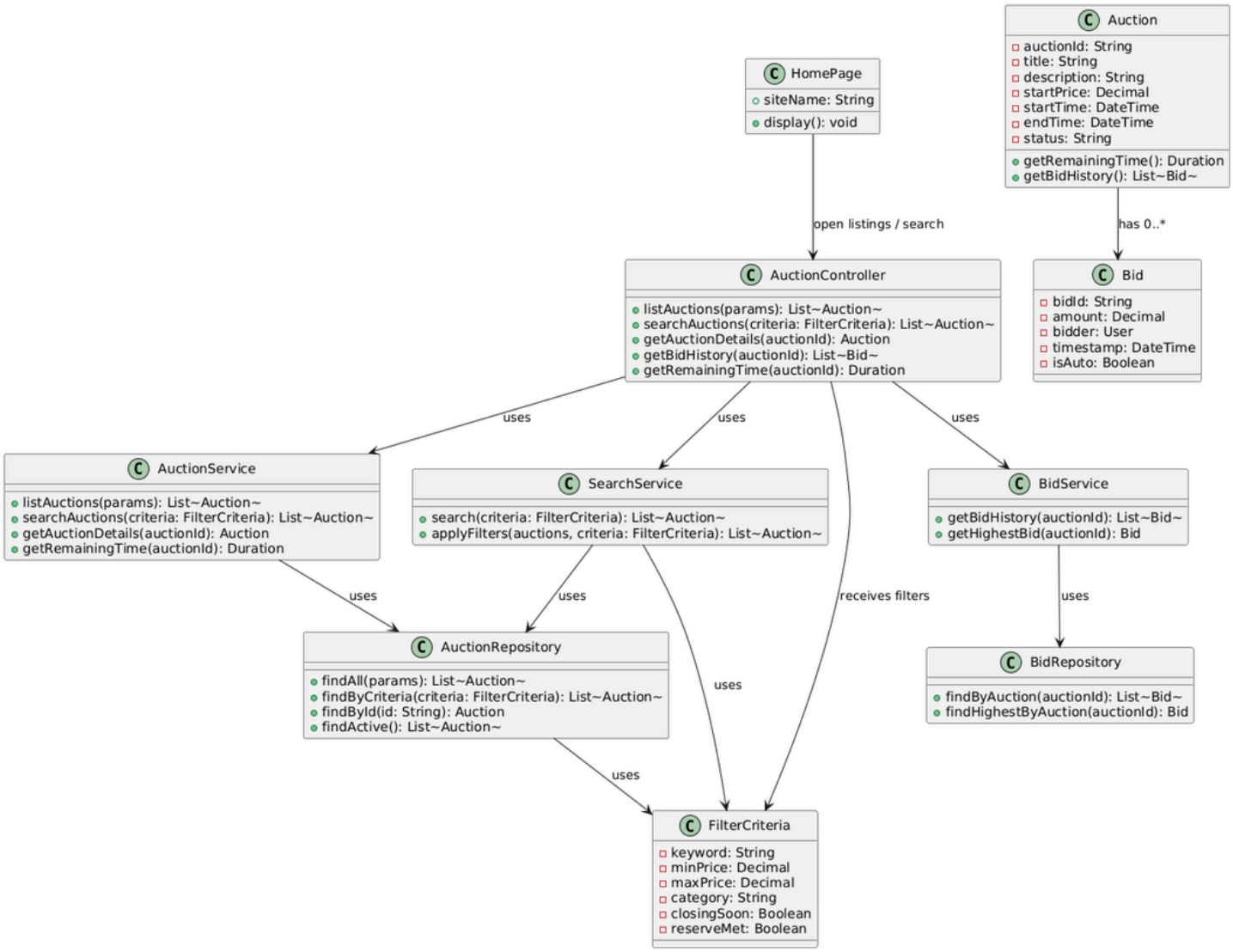
Features Implemented

1. **Auction listing page**
2. **Search by keyword, category**
3. **Filters (price, category, verified, live)**
4. **Bid history view**

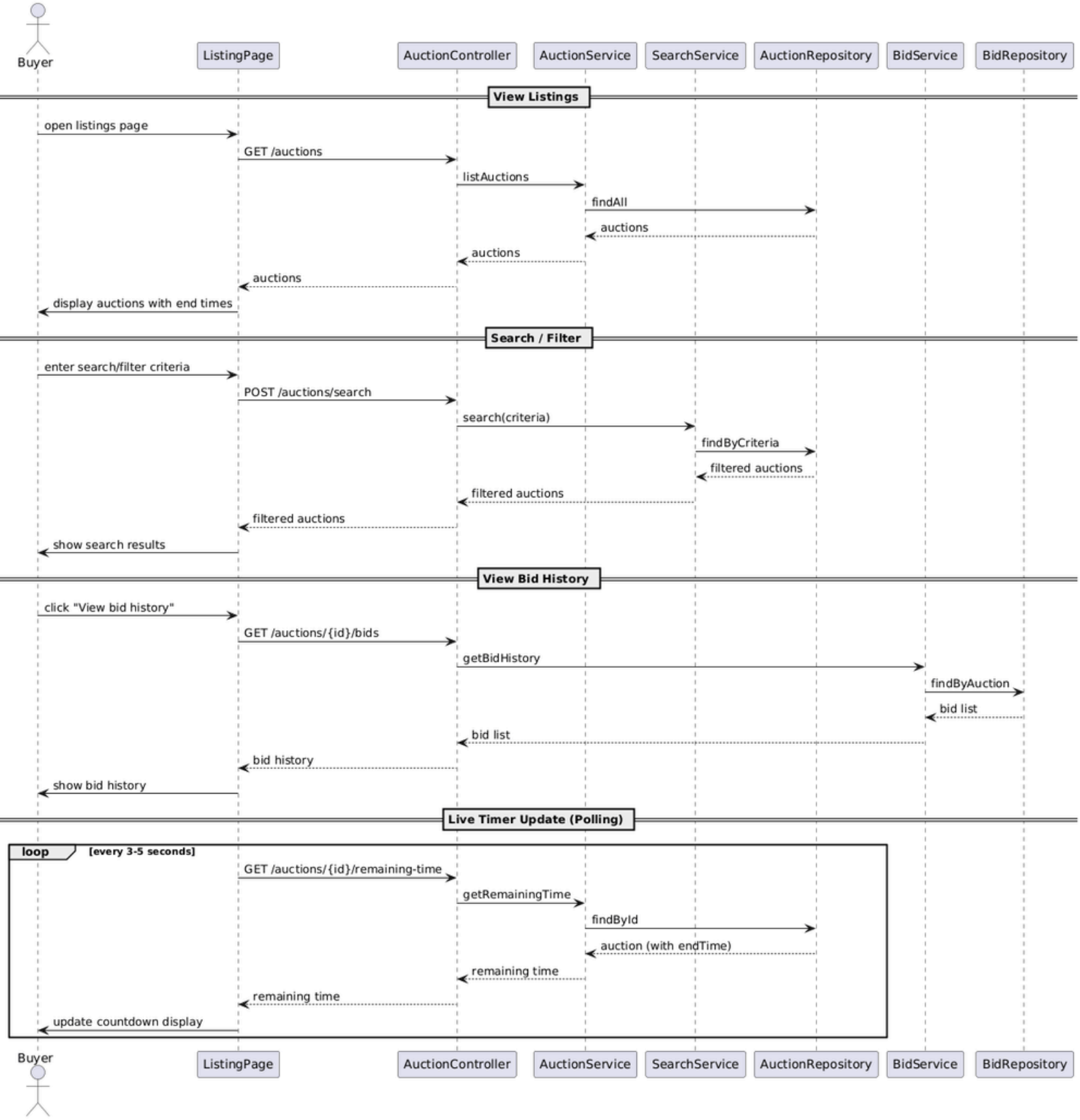
Sprint Goal

Enable buyers to easily browse, search, filter, and explore auction listings.

Class:


Auction
auctionId: String
title: String
description: String
startPrice: Decimal
startTime: DateTime
endTime: DateTime
status: String
getRemainingTime(): Duration
getBidHistory(): List~Bid~
HomePage
siteName: String
display(): void
open listings / search
has 0..*
AuctionController
listAuctions(params): List~Auction~
searchAuctions(criteria: FilterCriteria): List~Auction~
getAuctionDetails(auctionId): Auction
getBidHistory(auctionId): List~Bid~
getRemainingTime(auctionId): Duration
Bid
bidId: String
amount: Decimal
bidder: User
timestamp: DateTime
isAuto: Boolean
uses
uses
uses
AuctionService
listAuctions(params): List~Auction~
searchAuctions(criteria: FilterCriteria): List~Auction~
getAuctionDetails(auctionId): Auction
getRemainingTime(auctionId): Duration
SearchService
search(criteria: FilterCriteria): List~Auction~
applyFilters(auctions, criteria: FilterCriteria): List~Auction~
BidService
getBidHistory(auctionId): List~Bid~
getHighestBid(auctionId): Bid
uses
uses
receives filters
uses
AuctionRepository
findAll(params): List~Auction~
findByCriteria(criteria: FilterCriteria): List~Auction~
findById(id: String): Auction
findActive(): List~Auction~
BidRepository
findByAuction(auctionId): List~Bid~
findHighestByAuction(auctionId): Bid
uses
uses
FilterCriteria
keyword: String
minPrice: Decimal
maxPrice: Decimal
category: String
closingSoon: Boolean
reserveMet: Boolean

Sequence:



Sprint 4:Winner Payment, Payment Verification & Delivery Initialization

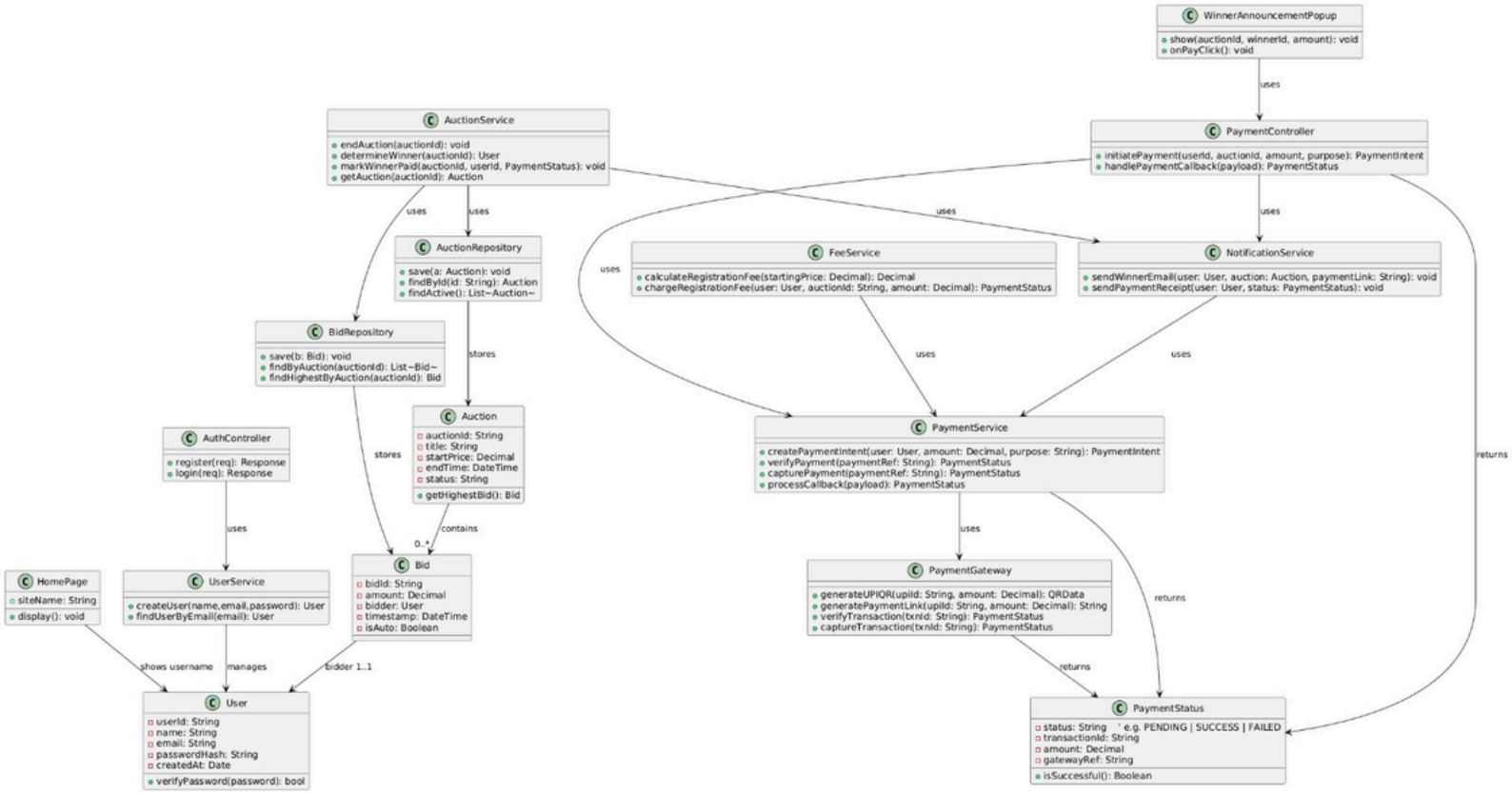
Features Implemented

1. **Winner is notified after auction ends (email + web popup)**
2. **Winner payment link + QR**
3. **Admin verifies winner payment**
4. **After payment success → Delivery order created**

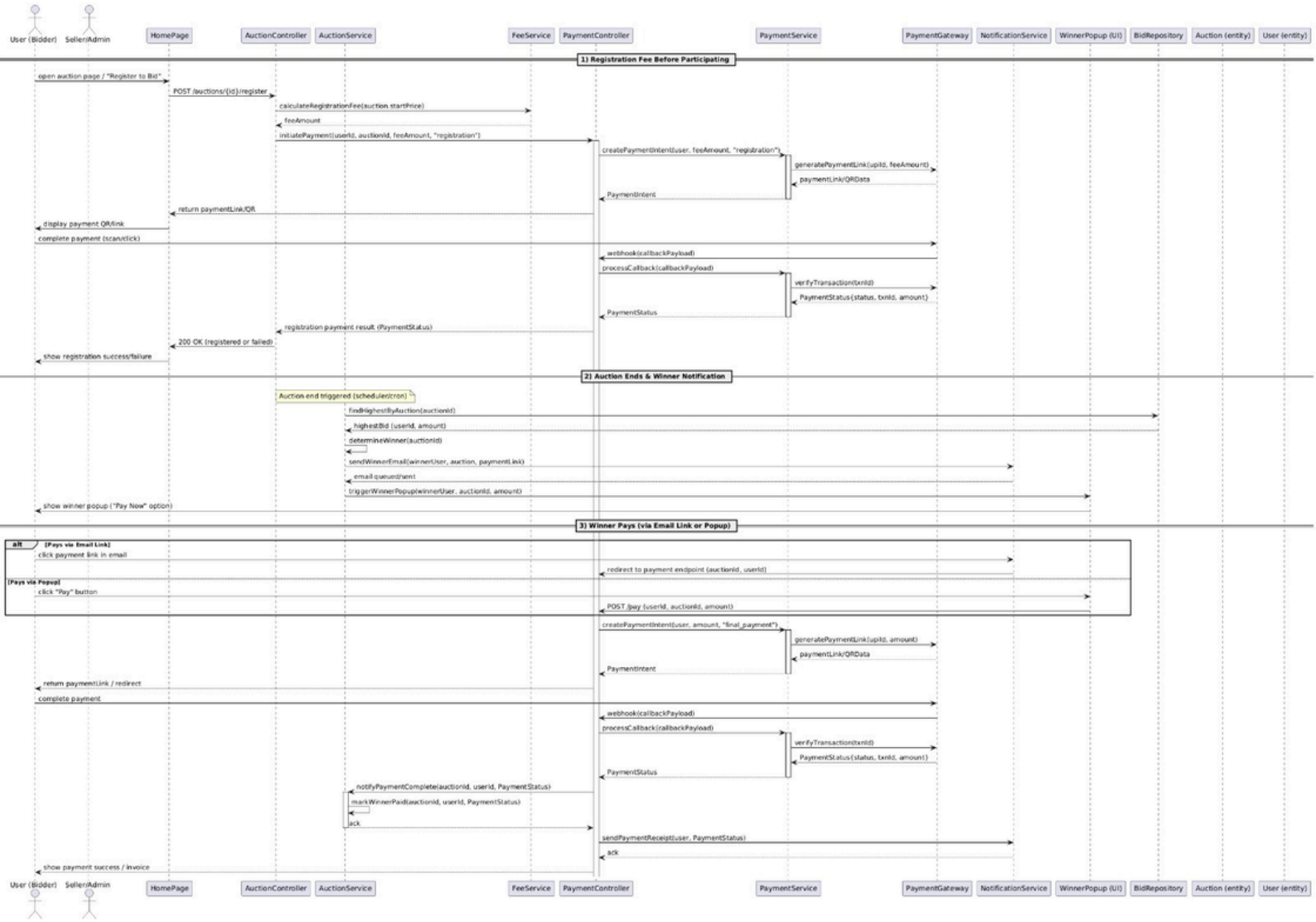
Sprint Goal

Complete the post-auction flow from winner selection to successful payment and delivery initiation.

Class:



Sequence:



Sprint 5:**Testing, Optimization & Production Deployment**

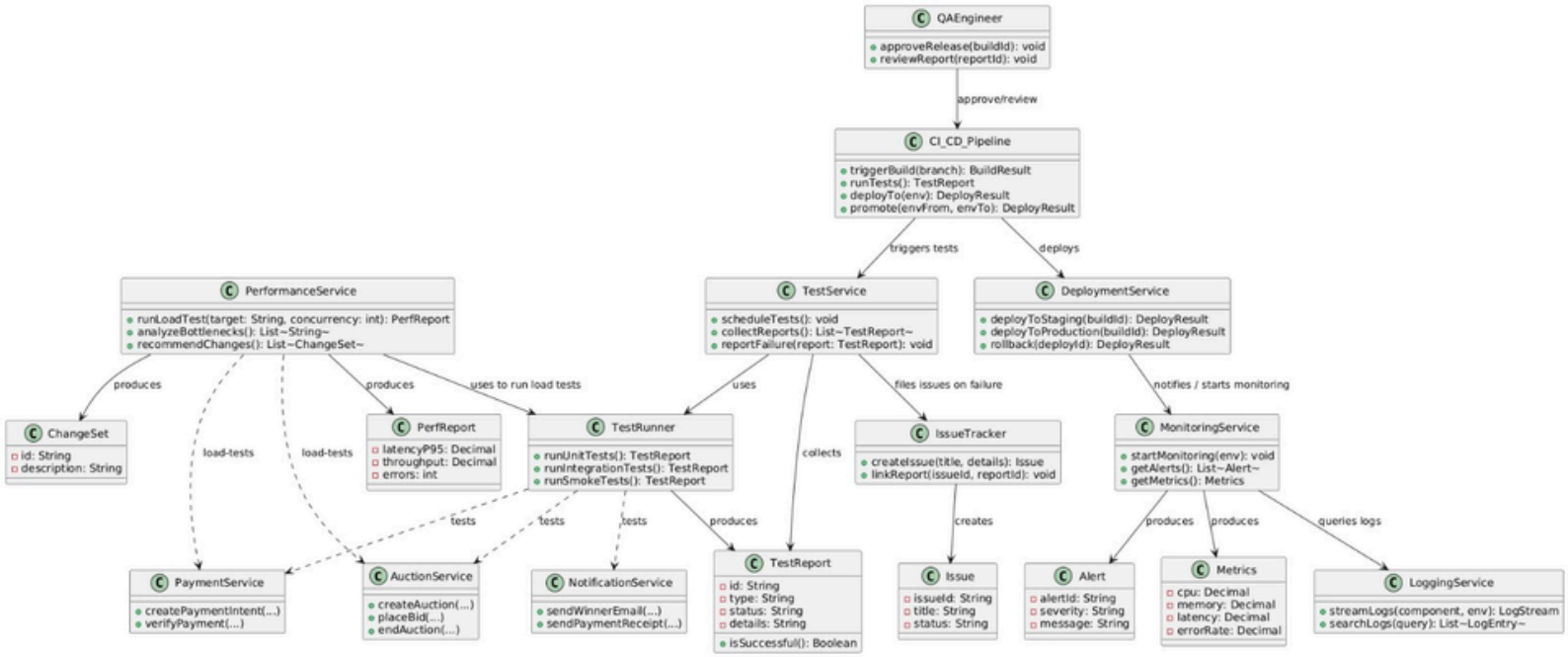
Features Implemented

1. **Full system testing (unit, integration, UAT)**
2. **Performance optimization**
3. **CI/CD pipeline setup**
4. **Deployment to staging & production**

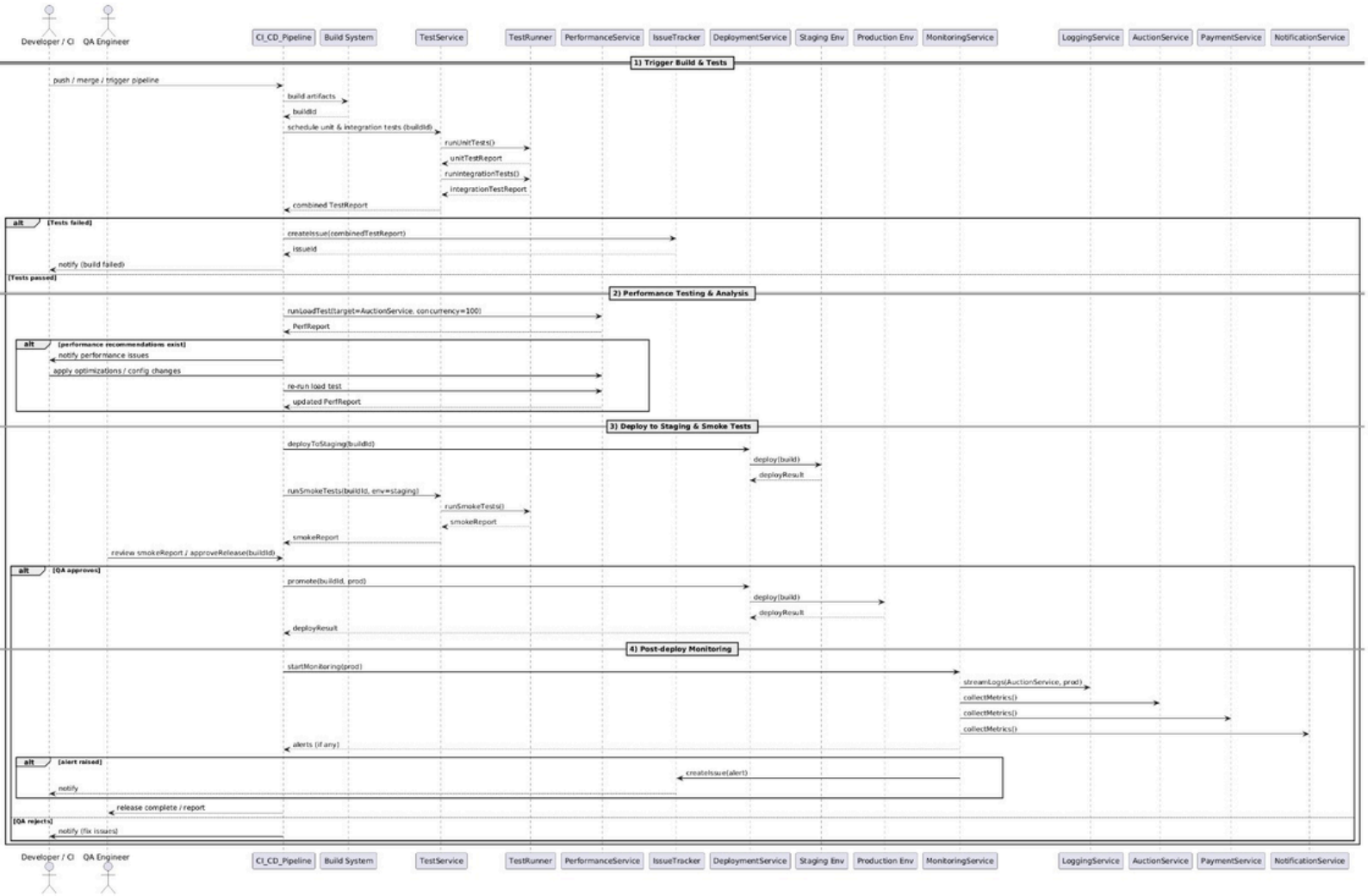
Sprint Goal

Ensure the system is reliable, optimized, and production-ready.

Class:


QAEngineer
approveRelease(buildId): void
reviewReport(reportId): void
approve/review
CI_CD_Pipeline
triggerBuild(branch): BuildResult
runTests(): TestReport
deployTo(env): DeployResult
promote(envFrom, envTo): DeployResult
triggers tests
deploys
PerformanceService
runLoadTest(target: String, concurrency: int): PerfReport
analyzeBottlenecks(): List~String~
recommendChanges(): List~ChangeSet~
TestService
scheduleTests(): void
collectReports(): List~TestReport~
reportFailure(report: TestReport): void
DeploymentService
deployToStaging(buildId): DeployResult
deployToProduction(buildId): DeployResult
rollback(deployId): DeployResult
produces
produces
uses to run load tests
uses
files issues on failure
notifies / starts monitoring
ChangeSet
id: String
description: String
load-tests
load-tests
PerfReport
latencyP95: Decimal
throughput: Decimal
errors: int
TestRunner
runUnitTests(): TestReport
runIntegrationTests(): TestReport
runSmokeTests(): TestReport
collects
IssueTracker
createIssue(title, details): Issue
linkReport(issueId, reportId): void
MonitoringService
startMonitoring(env): void
getAlerts(): List~Alert~
getMetrics(): Metrics
tests
tests
tests
produces
creates
produces
produces
queries logs
PaymentService
createPaymentIntent(...)
verifyPayment(...)
AuctionService
createAuction(...)
placeBid(...)
endAuction(...)
NotificationService
sendWinnerEmail(...)
sendPaymentReceipt(...)
TestReport
id: String
type: String
status: String
details: String
isSuccessful(): Boolean
Issue
issueId: String
title: String
status: String
Alert
alertId: String
severity: String
message: String
Metrics
cpu: Decimal
memory: Decimal
latency: Decimal
errorRate: Decimal
LoggingService
streamLogs(component, env): LogStream
searchLogs(query): List~LogEntry~

Sequence:


Developer / CI
QA Engineer
CI_CD_Pipeline
Build System
TestService
TestRunner
PerformanceService
IssueTracker
DeploymentService
Staging Env
Production Env
MonitoringService
LoggingService
AuctionService
PaymentService
NotificationService
1) Trigger Build & Tests
push / merge / trigger pipeline
build artifacts
buildId
schedule unit & integration tests (buildId)
runUnitTests()
unitTestReport
runIntegrationTests()
integrationTestReport
combined TestReport
alt
[Tests failed]
createIssue(combinedTestReport)
issueId
notify (build failed)
[Tests passed]
2) Performance Testing & Analysis
runLoadTest(target=AuctionService, concurrency=100)
PerfReport
alt
[performance recommendations exist]
notify performance issues
apply optimizations / config changes
re-run load test
updated PerfReport
3) Deploy to Staging & Smoke Tests
deployToStaging(buildId)
deploy(build)
deployResult
runSmokeTests(buildId, env=staging)
runSmokeTests()
smokeReport
smokeReport
review smokeReport / approveRelease(buildId)
alt
[QA approves]
promote(buildId, prod)
deploy(build)
deployResult
deployResult
4) Post-deploy Monitoring
startMonitoring(prod)
streamLogs(AuctionService, prod)
collectMetrics()
collectMetrics()
collectMetrics()
alerts (if any)
alt
[alert raised]
createIssue(alert)
notify
release complete / report
[QA rejects]
notify (fix issues)

Sprint 6:Admin Dashboard, Watchlist & Seller Rating

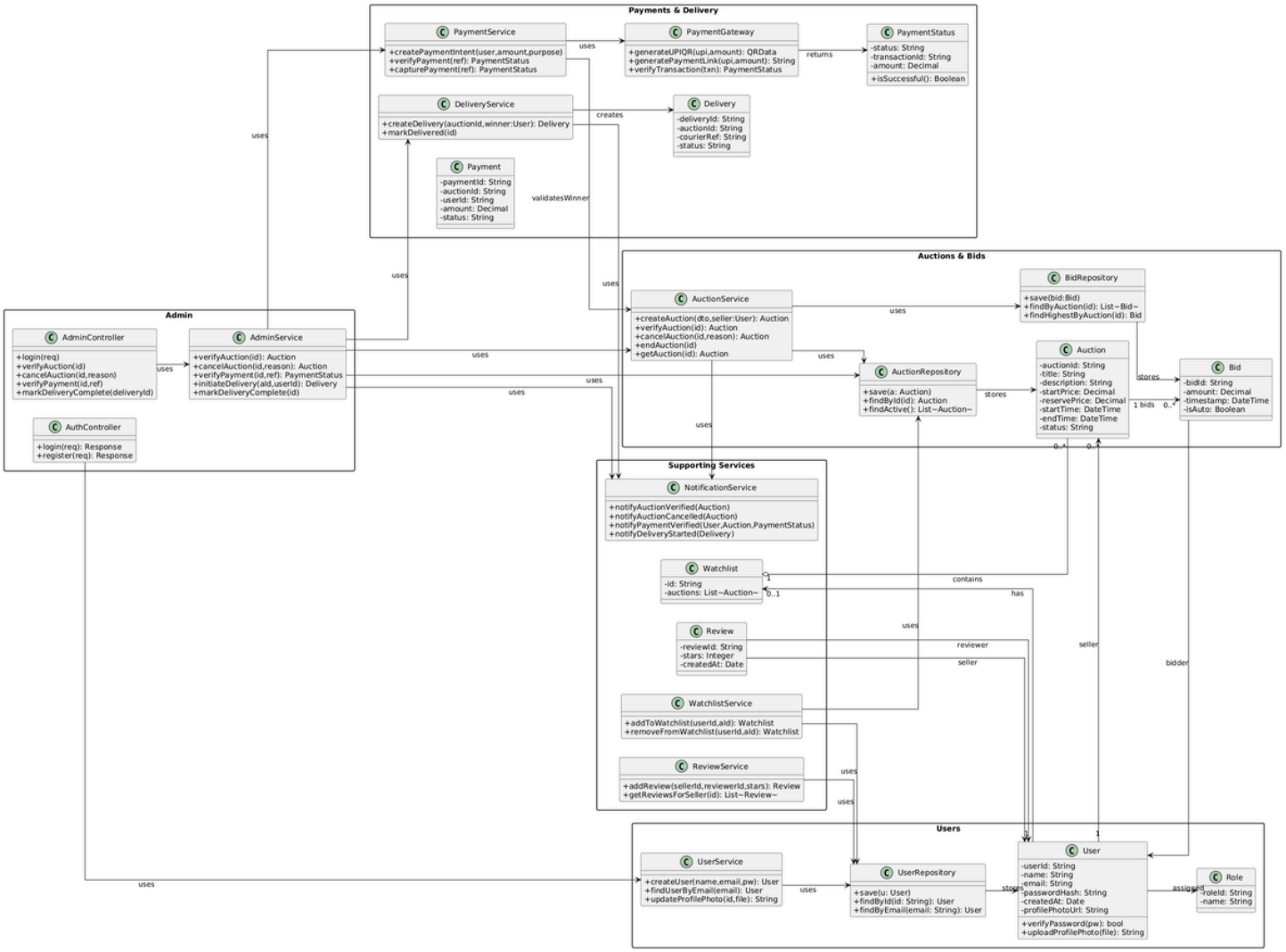
Features Implemented

1. **Admin login**
2. **Admin verifies or cancels auctions**
3. **Admin verifies payments**
4. **Delivery completion confirmation**
5. **User watchlist (add/remove)**
6. **Seller review rating (1–5 stars)**
7. **User profile photo upload**
8. **Auction images stored in Cloudinary**

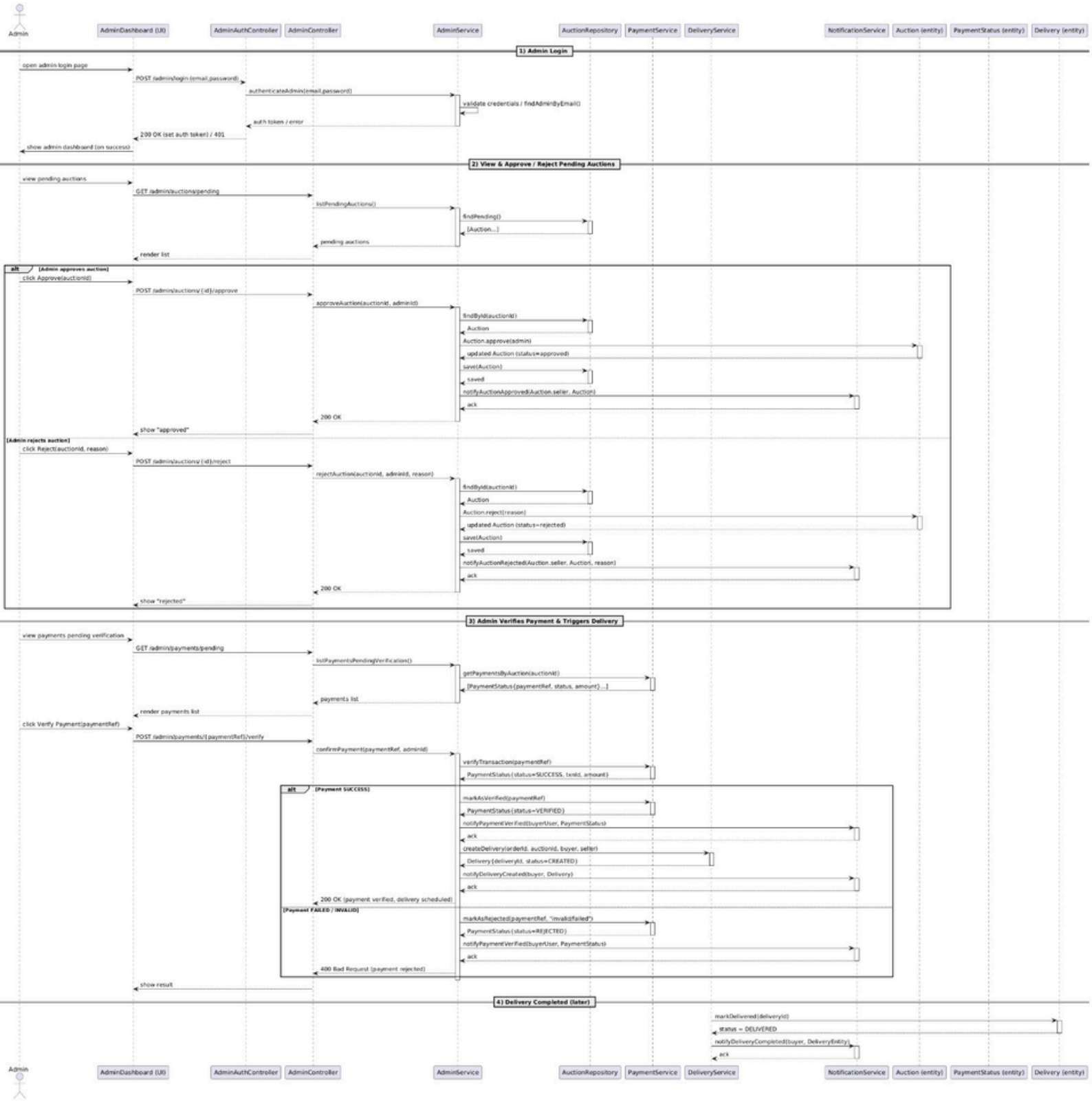
Sprint Goal

Provide administrative oversight and improve user engagement through watchlists and seller trust ratings.

Class:


Payments & Delivery
PaymentService
+createPaymentIntent(user,amount,purpose)
+verifyPayment(ref): PaymentStatus
+capturePayment(ref): PaymentStatus
PaymentGateway
+generateUPIQR(upi,amount): QRData
+generatePaymentLink(upi,amount): String
+verifyTransaction(txn): PaymentStatus
PaymentStatus
-status: String
-transactionId: String
-amount: Decimal
+isSuccessful(): Boolean
DeliveryService
+createDelivery(auctionId,winner:User): Delivery
+markDelivered(id)
Delivery
-deliveryId: String
-auctionId: String
-courierRef: String
-status: String
Payment
-paymentId: String
-auctionId: String
-userId: String
-amount: Decimal
-status: String
uses
creates
returns
validatesWinner
Auctions & Bids
AuctionService
+createAuction(dto,seller:User): Auction
+verifyAuction(id): Auction
+cancelAuction(id,reason): Auction
+endAuction(id)
+getAuction(id): Auction
BidRepository
+save(bid:Bid)
+findByAuction(id): List~Bid~
+findHighestByAuction(id): Bid
AuctionRepository
+save(a: Auction)
+findById(id): Auction
+findActive(): List~Auction~
Auction
-auctionId: String
-title: String
-description: String
-startPrice: Decimal
-reservePrice: Decimal
-startTime: DateTime
-endTime: DateTime
-status: String
Bid
-bidId: String
-amount: Decimal
-timestamp: DateTime
-isAuto: Boolean
stores
1 bids
0..*
Admin
AdminController
+login(req)
+verifyAuction(id)
+cancelAuction(id,reason)
+verifyPayment(id,ref)
+markDeliveryComplete(deliveryId)
AdminService
+verifyAuction(id): Auction
+cancelAuction(id,reason): Auction
+verifyPayment(id,ref): PaymentStatus
+initiateDelivery(aId,userId): Delivery
+markDeliveryComplete(id)
AuthController
+login(req): Response
+register(req): Response
Supporting Services
NotificationService
+notifyAuctionVerified(Auction)
+notifyAuctionCancelled(Auction)
+notifyPaymentVerified(User,Auction,PaymentStatus)
+notifyDeliveryStarted(Delivery)
Watchlist
-id: String
-auctions: List~Auction~
contains
has
0..1
Review
-reviewId: String
-stars: Integer
-createdAt: Date
reviewer
seller
bidder
WatchlistService
+addToWatchlist(userId,aId): Watchlist
+removeFromWatchlist(userId,aId): Watchlist
ReviewService
+addReview(sellerId,reviewerId,stars): Review
+getReviewsForSeller(id): List~Review~
Users
UserService
+createUser(name,email,pw): User
+findUserByEmail(email): User
+updateProfilePhoto(id,file): String
UserRepository
+save(u: User)
+findById(id: String): User
+findByEmail(email: String): User
User
-userId: String
-name: String
-email: String
-passwordHash: String
-createdAt: Date
-profilePhotoUrl: String
+verifyPassword(pw): bool
+uploadProfilePhoto(file): String
Role
-roleId: String
-name: String
stores
assigned

Sequence:


Admin
AdminDashboard (UI)
AdminAuthController
AdminController
AdminService
AuctionRepository
PaymentService
DeliveryService
NotificationService
Auction (entity)
PaymentStatus (entity)
Delivery (entity)
1) Admin Login
open admin login page
POST /admin/login (email,passwordd)
authenticateAdmin(email,password)
validate credentials / findAdminByEmail()
auth token / error
200 OK (set auth token) / 401
show admin dashboard (on success)
2) View & Approve / Reject Pending Auctions
view pending auctions
GET /admin/auctions/pending
listPendingAuctions()
findPending()
[Auction...]
pending auctions
render list
alt
[Admin approves auction]
click Approve(auctionId)
POST /admin/auctions/{id}/approve
approveAuction(auctionId, adminId)
findById(auctionId)
Auction
Auction.approve(admin)
updated Auction (status=approved)
save(Auction)
saved
notifyAuctionApproved(Auction.seller, Auction)
ack
200 OK
show "approved"
[Admin rejects auction]
click Reject(auctionId, reason)
POST /admin/auctions/{id}/reject
rejectAuction(auctionId, adminId, reason)
findById(auctionId)
Auction
Auction.reject(reason)
updated Auction (status=rejected)
save(Auction)
saved
notifyAuctionRejected(Auction.seller, Auction, reason)
ack
200 OK
show "rejected"
3) Admin Verifies Payment & Triggers Delivery
view payments pending verification
GET /admin/payments/pending
listPaymentsPendingVerification()
getPaymentsByAuction(auctionId)
[PaymentStatus{paymentRef, status, amount}...]
payments list
render payments list
click Verify Payment(paymentRef)
POST /admin/payments/{paymentRef}/verify
confirmPayment(paymentRef, adminId)
verifyTransaction(paymentRef)
PaymentStatus{status=SUCCESS, txId, amount}
alt
[Payment SUCCESS]
markAsVerified(paymentRef)
PaymentStatus{status=VERIFIED}
notifyPaymentVerified(buyerUser, PaymentStatus)
ack
createDelivery(orderId, auctionId, buyer, seller)
Delivery{deliveryId, status=CREATED}
notifyDeliveryCreated(buyer, Delivery)
ack
200 OK (payment verified, delivery scheduled)
[Payment FAILED / INVALID]
markAsRejected(paymentRef, "invalid/failed")
PaymentStatus{status=REJECTED}
notifyPaymentVerified(buyerUser, PaymentStatus)
ack
400 Bad Request (payment rejected)
show result
4) Delivery Completed (later)
markDelivered(deliveryId)
status = DELIVERED
notifyDeliveryCompleted(buyer, DeliveryEntity)
ack